



University of Stuttgart  
Germany

# PDDL

Modelling classical planning problems

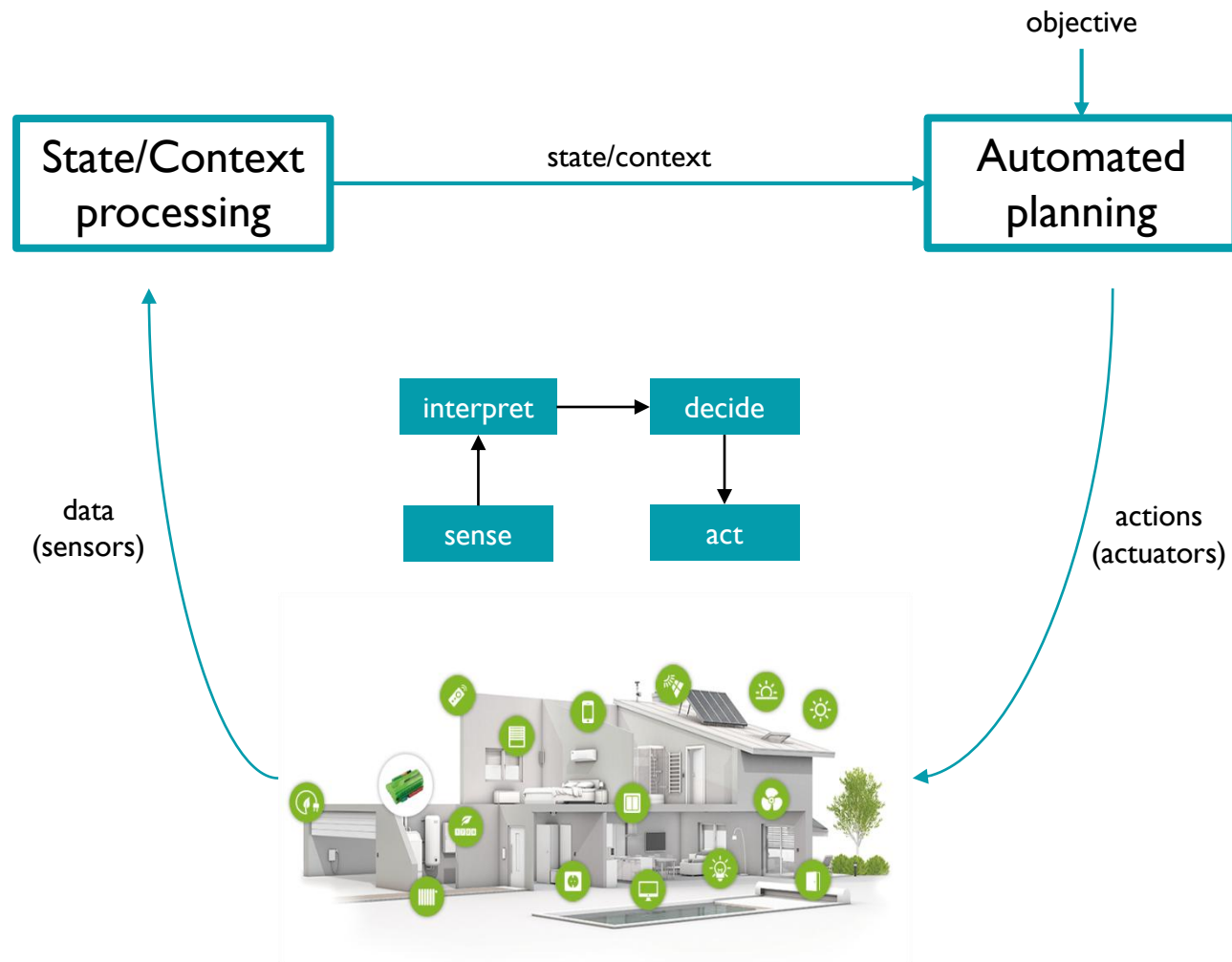
Ilche Georgievski

[ilche.georgievski@iaas.uni-stuttgart.de](mailto:ilche.georgievski@iaas.uni-stuttgart.de)

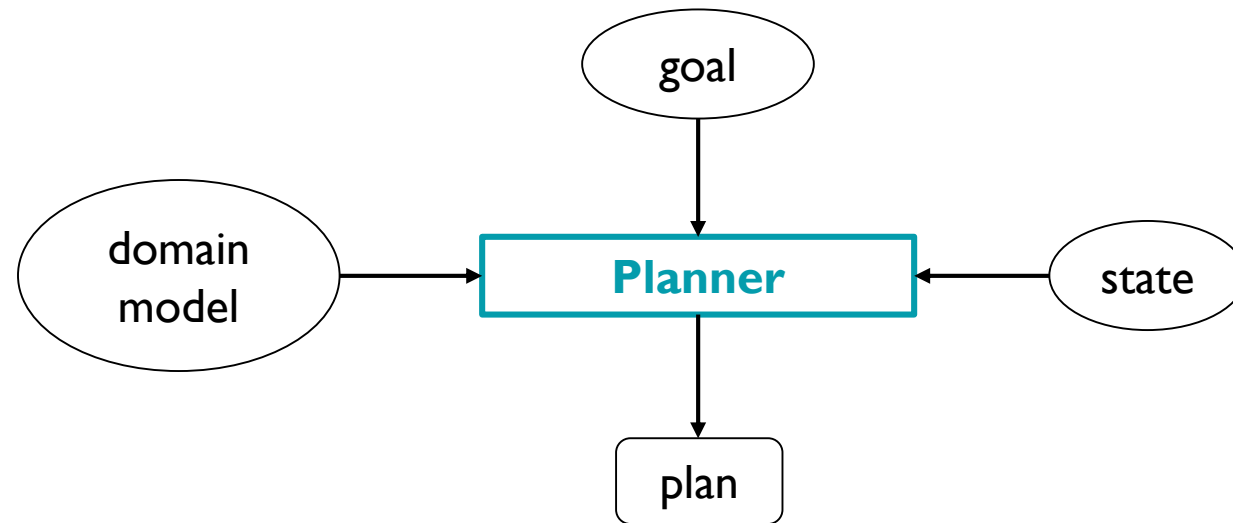
Room: U38.03

2026

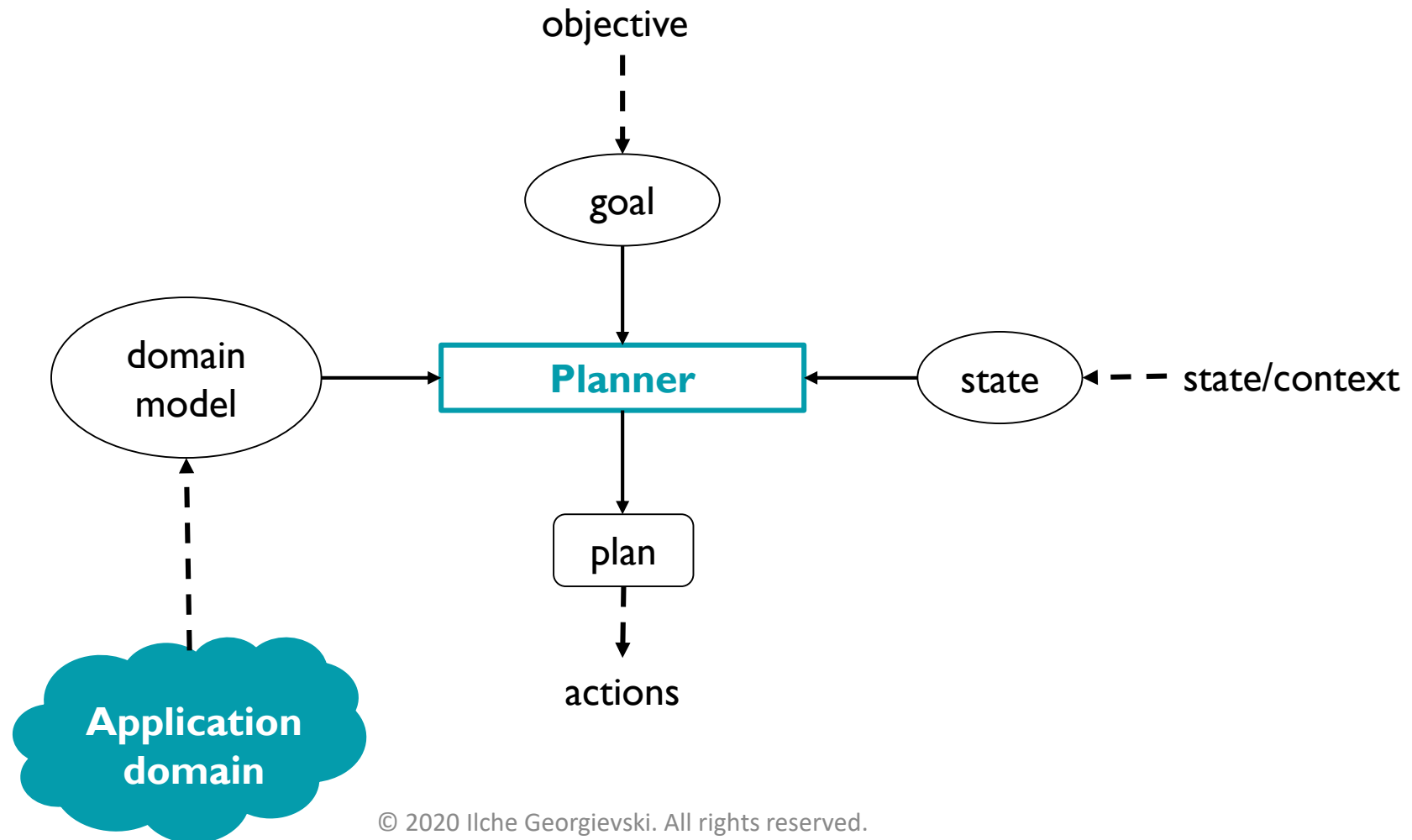
Summer



# General view



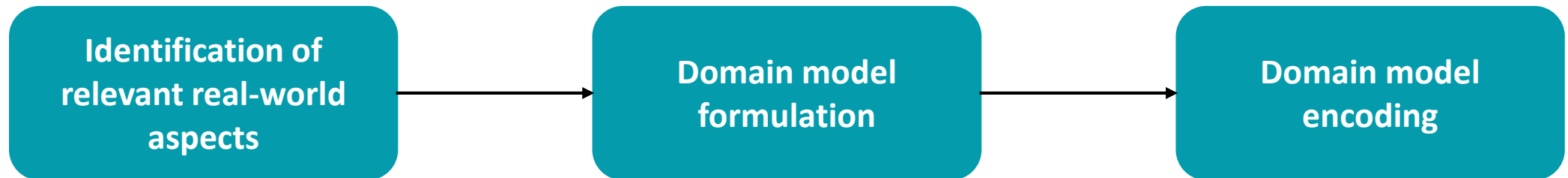
# General view



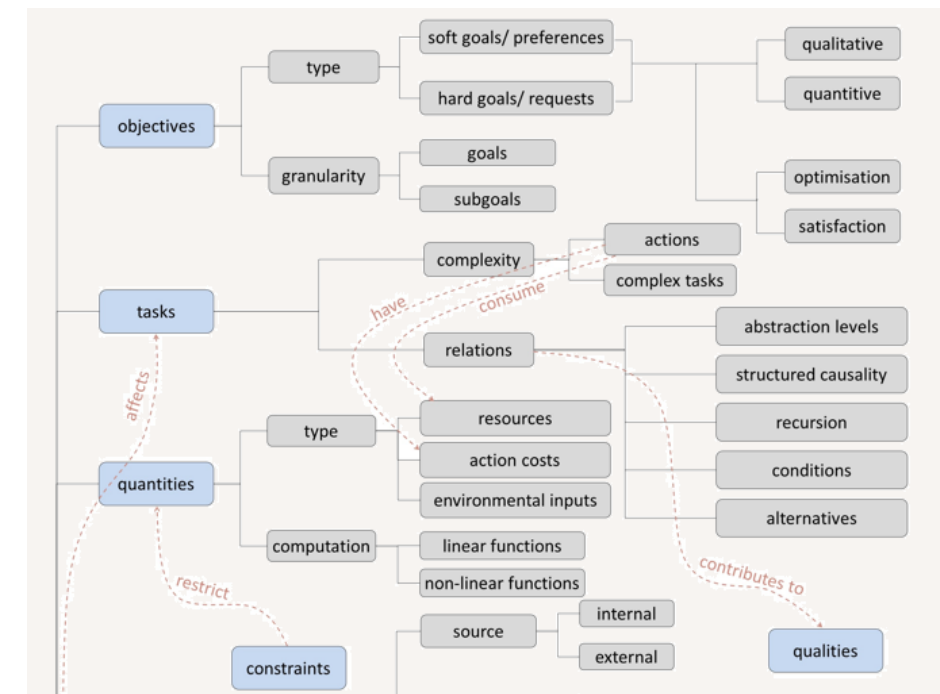
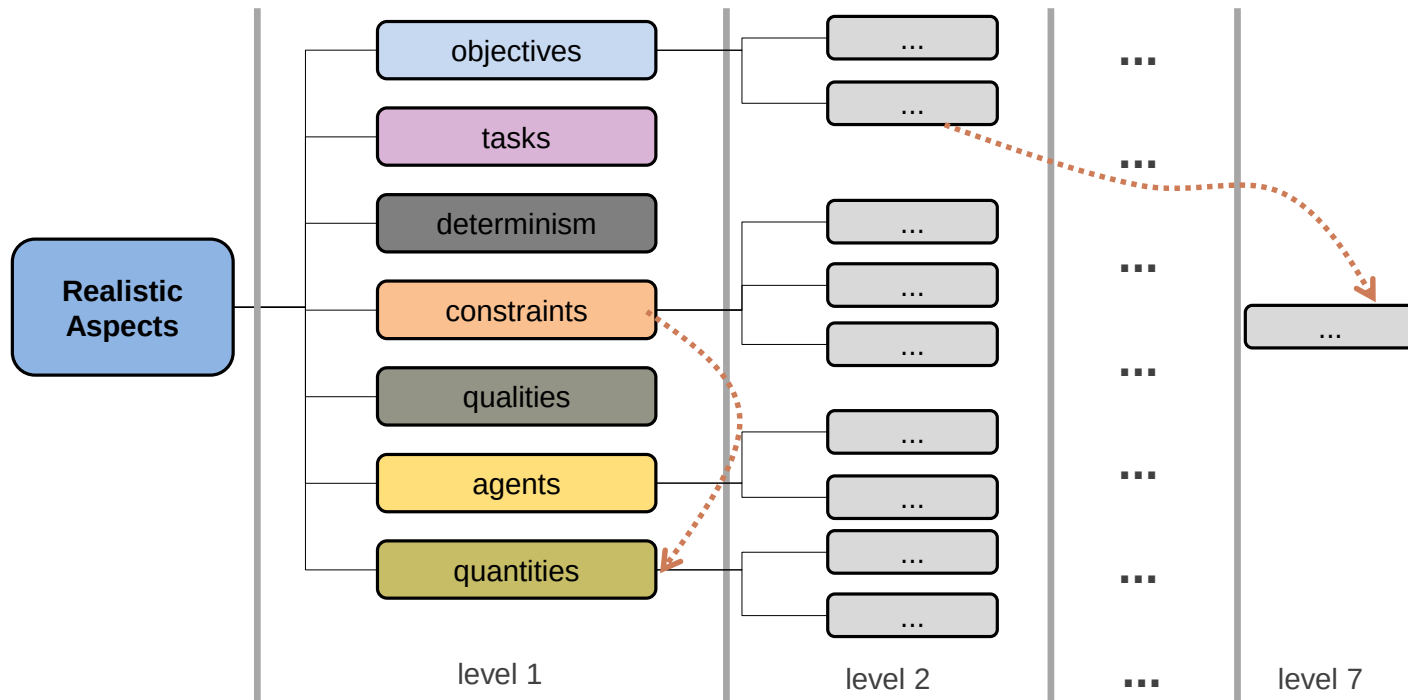
# Domain model

- Knowledge about the domain of interest
- It can be developed, tested, debugged and validated independently from planners
- It is influenced by *what planners can handle*
- It may be useful for more purposes (than only planning)

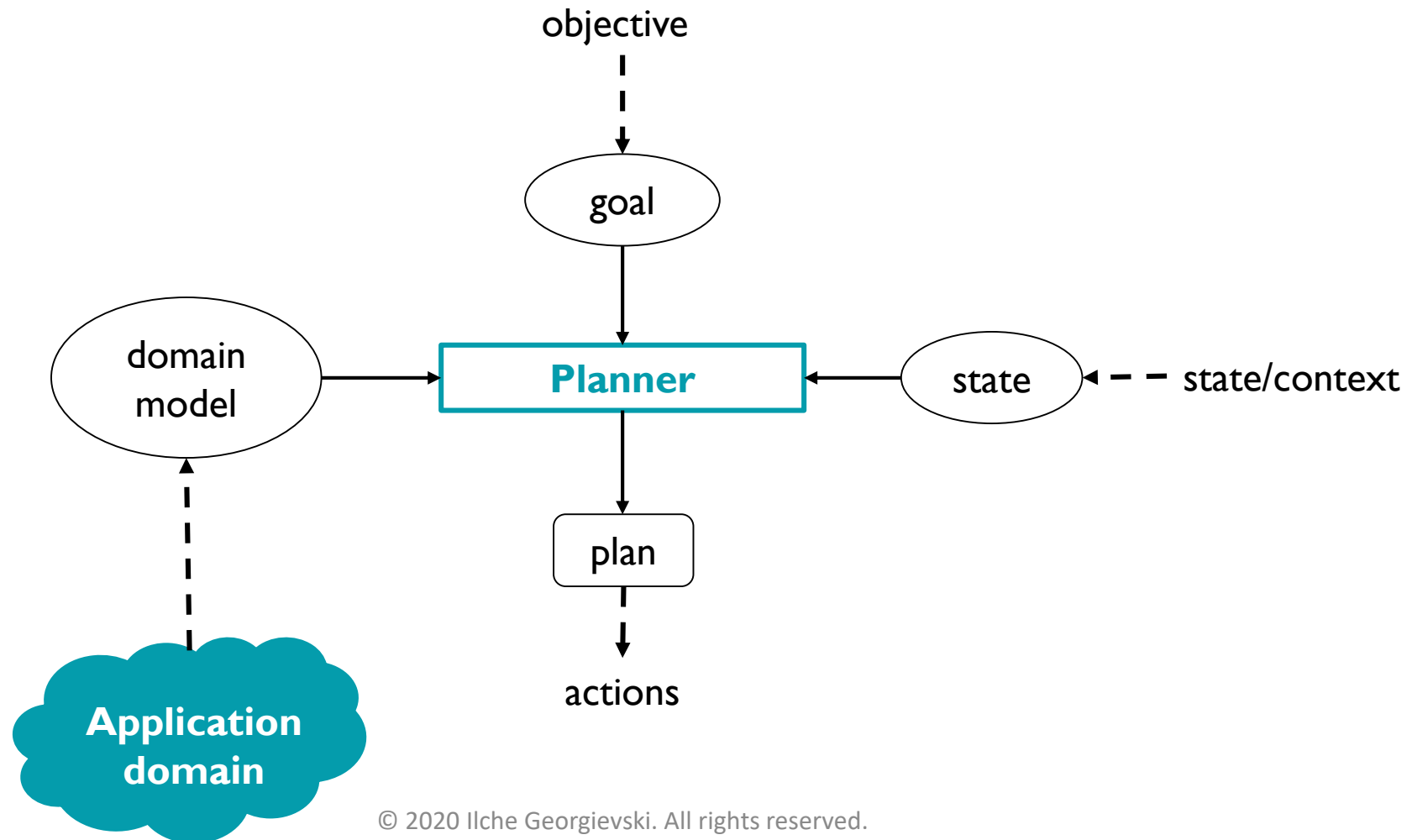
# Engineering Domain model



# Engineering Domain model

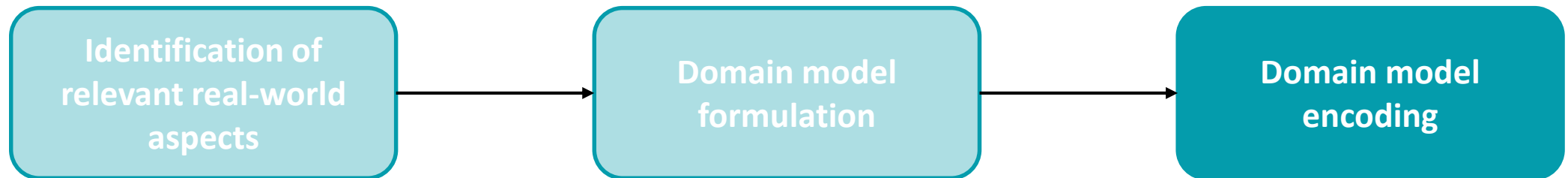


# General view





# Engineering Domain model



Planning Domain Definition Language

**PDDL**

# Intro

- Why?
  - *De-facto* standard input language for classical planning since 1998
- Its development is mainly driven by the International Planning Competition (IPC)
- 5 so-called levels
  - Level 1: STRIPS and ADL
  - Level 2: Numerical planning
  - Level 3: Temporal planning
  - Level 4: Derived predicated and timed initial literals
  - Level 5: Soft goals and trajectory constraints

# Intro

- First-order logic language
- Uses schematic encodings
- Lisp-like syntax

# Schematic encoding

- Description of the world in a schematic way relative to a set of objects
  - Set of objects is finite
  - Use variables that range over objects

# Schemas

- For predicates and actions
- **Arity** is the number of variables in predicates/actions
  - In practice, most domains have a maximum arity of 3
- **Grounding** is instantiating variables
  - Exponential in the action and predicate arity
- Most planners translate the schematic input into a propositional encoding in a preprocessing step by grounding variables in all possible ways

PDDL on an example

# **TARS, THE GRIPPER**

# TARS, the Gripper

- A gripping planning problem with four balls\*
- TARS, a smart home robot, can move between boy's room and girl's room, and can pick up or drop balls with either of his two arms. Initially, all balls and TARS are in the boy's room. We want the balls to be in the girl's room.



# Schematic encoding

- Move *robot* from  $r_1$  to  $r_2$ :
    - $r_1 \in \{girlRoom, boyroom\}$
    - $r_2 \in \{girlRoom, boyroom\}$
    - $robot \in \{TARS1, TARS2\}$
  - Pre:  $at(robot, r_1)$
  - Add:  $at(robot, r_2)$
  - Del:  $at(robot, r_1)$
- **Artiy** = 3
  - Grounding  $\rightarrow 2^3$

# Schematic encoding

- Gounding

- Pre: at(*TARS1*, *girlRoom*)
- Add: at(*TARS1*, *boyroom*)
- Del: at(*TARS1*, *girlRoom*)
  
- Pre: at(*TARS1*, *boyroom*)
- Add: at(*TARS1*, *girlRoom*)
- Del: at(*TARS1*, *boyroom*)
  
- Pre: at(*TARS1*, *boyroom*)
- Add: at(*TARS1*, *boyroom*)
- Del: at(*TARS1*, *boyroom*)
  
- Pre: at(*TARS1*, *girlRoom*)
- Add: at(*TARS1*, *girlRoom*)
- Del: at(*TARS1*, *girlRoom*)

- Pre: at(*TARS2*, *girlRoom*)
- Add: at(*TARS2*, *boyroom*)
- Del: at(*TARS2*, *girlRoom*)
  
- Pre: at(*TARS2*, *boyroom*)
- Add: at(*TARS2*, *girlRoom*)
- Del: at(*TARS2*, *boyroom*)
  
- Pre: at(*TARS2*, *boyroom*)
- Add: at(*TARS2*, *boyroom*)
- Del: at(*TARS2*, *boyroom*)
  
- Pre: at(*TARS2*, *girlRoom*)
- Add: at(*TARS2*, *girlRoom*)
- Del: at(*TARS2*, *girlRoom*)

# Components

- **Objects** are things in the world we are interested in
- **Types** are strings describing the types of objects being referred to
- **Predicates** are properties of objects which can be *true* and *false*
- **Actions** describe ways of changing the state of the world
- **Initial state** is the state of the world in which we start
- **Goal description** contains the things that we want to be true

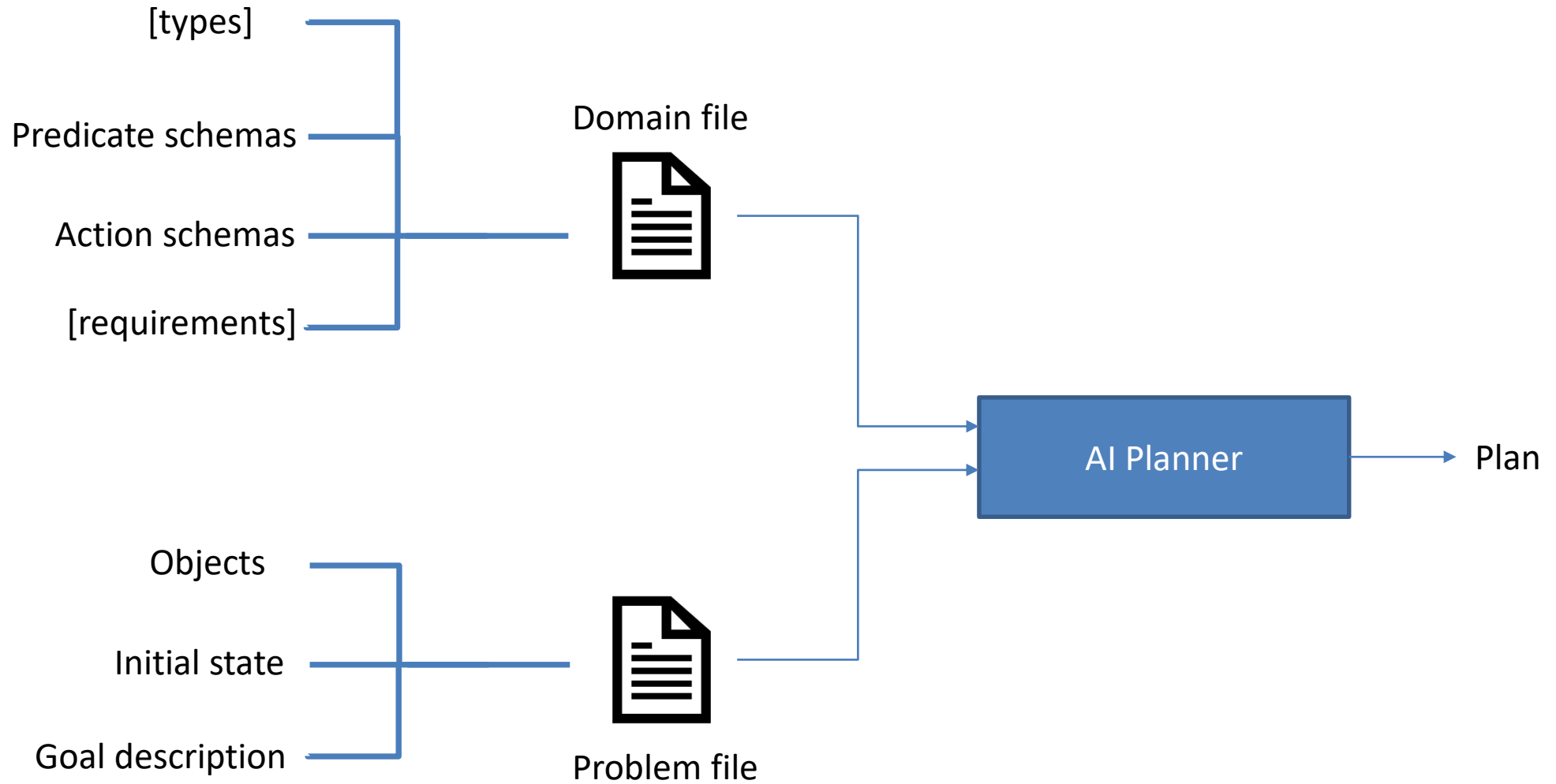
# TARS, the Gripper

- **Objects:**
  - boy's room, girl's room, four balls, TARS' arms
- **Types:**
  - room, ball, arm
- **Predicates:**
  - Is x a room? Is x a ball? Is ball x inside room y? Is TARS' arm x empty? [...]
- **Actions:**
  - TARS can move between the rooms, pick up a ball or drop a ball
- **Initial state:**
  - all balls and TARS are in the boy's room. Both TARS' arms are empty. [...]
- **Goal description:**
  - all balls must be in the girl's room

# Model in two files

- Planner's input separated into two PDDL files
  - Domain file for types, predicates and actions
  - Problem file for objects, initial state and goal
- Files have a .pddl extension

# Model in two files



# Domain file

- Basic structure:

```
(define (domain <domain-name>)  
  <PDDL code for requirements>  
  <PDDL code for types>  
  <PDDL code for predicates>  
  <PDDL code for action schemas>  
)
```

- **<domain-name>** is a string that identifies the application domain

# Requirements

- Declare which features are used
  - What should a planner handle
  - What can a planner handle
- If not specified, the default requirement is *:strips*

| Requirement                | Description                                                                                                                               |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| :strips                    | Basic STRIPS-style add and delete lists                                                                                                   |
| :typing                    | Allow type names in declarations of variables                                                                                             |
| :negative-preconditions    | Allow not in goal descriptions                                                                                                            |
| :disjunctive-preconditions | Allow or in goal descriptions                                                                                                             |
| :equality                  | Support = as a built-in predicate                                                                                                         |
| :existential-preconditions | Allow exists in goal descriptions                                                                                                         |
| :universal-preconditions   | Allow forall in goal descriptions                                                                                                         |
| :quantified-preconditions  | = :existential-preconditions + :universal-preconditions                                                                                   |
| :conditional-effects       | Allow when in action effects                                                                                                              |
| :fluents                   | Allow function definitions, arithmetic preconditions and effects with assignment operators                                                |
| :adl                       | = :strips + :typing + :negative-preconditions + :disjunctive-preconditions + :equality + :quantified-preconditions + :conditional-effects |
| ...                        |                                                                                                                                           |



# Types

## In plain text

- room
- ball
- arm

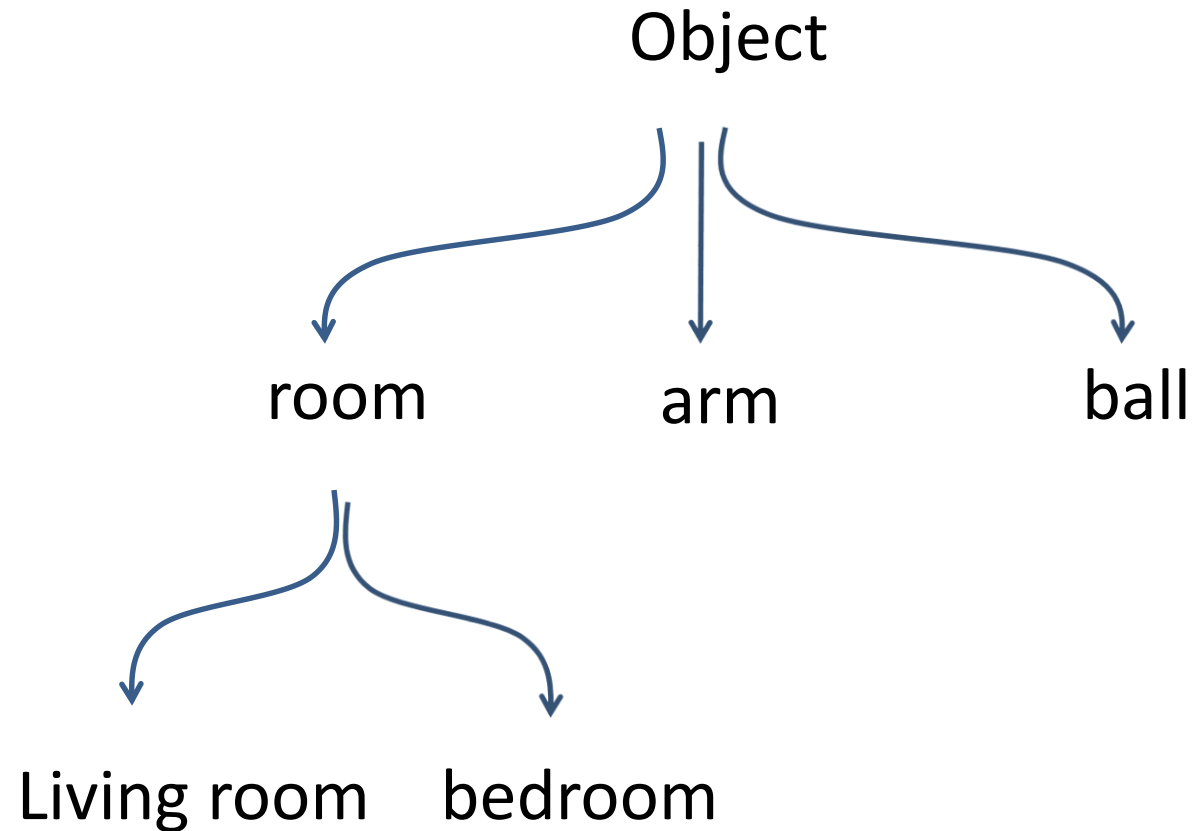
## In PDDL

```
(:types room ball arm - object)
```

# Types- type hierarchy

In PDDL

```
(:types room ball arm - object  
        livingRoom bedroom -room)
```



# Predicates

|                                         |                                                          |
|-----------------------------------------|----------------------------------------------------------|
| <code>ROOM(<math>x</math>)</code>       | true iff $x$ is a room                                   |
| <code>BALL(<math>y</math>)</code>       | true iff $y$ is a ball                                   |
| <code>ARM(<math>z</math>)</code>        | true iff $z$ is an arm                                   |
| <code>at-robot(<math>x</math>)</code>   | true iff $x$ is a room and TARS is in $x$                |
| <code>at-ball(<math>y, x</math>)</code> | true iff $y$ is a ball, $x$ is a room, and $y$ is in $x$ |
| <code>free(<math>z</math>)</code>       | true iff $z$ is an arm and $z$ does not hold a ball      |
| <code>carry(<math>z, y</math>)</code>   | true iff $z$ is an arm, $y$ is a ball, and $z$ holds $y$ |

# Predicates in PDDL

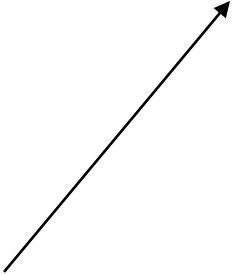
## Without typing

```
(:predicates
  (ROOM ?x)
  (BALL ?y)
  (ARM ?z)
  (at-robot ?x)
  (at-ball ?y ?x)
  (free ?z)
  (carry ?z ?y))
```

## With typing

```
(:predicates
  (at-robot ?x - room)
  (at-ball ?y - ball ?x - room)
  (free ?z - arm)
  (carry ?z - arm ?y - ball))
```

syntactic sugar



# Movement action

**Description:** TARS can move from  $x$  to  $y$

**Precondition:**  $\text{ROOM}(x)$ ,  $\text{ROOM}(y)$  and  $\text{at-robot}(x)$  are true.

**Effect:**  $\text{at-robot}(y)$  becomes true.  
 $\text{at-robot}(x)$  becomes false.

# Movement action in PDDL

## Without typing

```
(:action move
  :parameters (?x ?y)
  :precondition (and (ROOM ?x)
                    (ROOM ?y)
                    (at-robot ?x))
  :effect (and (at-robot ?y)
              (not (at-robot ?x))))
```

## With typing

```
(:action move
  :parameters (?x ?y - room)
  :precondition (at-robot ?x)
  :effect (and (at-robot ?y)
              (not (at-robot ?x))))
```

# Pick-up action

**Description:** TARS can pick up  $x$  with  $z$  in  $y$

**Precondition:**  $BALL(x), ROOM(y), ARM(z), at-ball(x, y),$   
 $at-robot(y)$  and  $free(z)$  are true.

**Effect:**  $carry(z, x)$  becomes true.  
 $at-ball(x, y)$  and  $free(z)$  become false.

# Pick-up action in PDDL

## Without typing

```
(:action pick-up
  :parameters (?x ?y ?z)
  :precondition (and (BALL ?x)
                     (ROOM ?y)
                     (ARM ?z)
                     (at-ball ?x ?y)
                     (at-robot ?y)
                     (free ?z))
  :effect (and (carry ?z ?x)
              (not (at-ball ?x ?y))
              (not (free ?z))))
```

## With typing

```
(:action pick-up
  :parameters (?x - ball ?y - room ?z - arm)
  :precondition (and (at-ball ?x ?y)
                     (at-robot ?y)
                     (free ?z))
  :effect (and (carry ?z ?x)
              (not (at-ball ?x ?y))
              (not (free ?z))))
```



# Drop action

**Description:** TARS can drop  $x$  in  $y$  from  $z$

Preconditions and effects similar to the pick-up action.

## PDDL without typing

```
(:action drop
  :parameters (?x ?y ?z)
  :precondition (and (BALL ?x)
                     (ROOM ?y)
                     (ARM ?z)
                     (carry ?z ?x)
                     (at-robot ?y))
  :effect (and (at-ball ?x ?y)
               (free ?z)
               (not (carry ?z ?x))))
```

## PDDL with typing

```
(:action drop
  :parameters (?x - ball ?y - room ?z - arm)
  :precondition (and (carry ?z ?x)
                     (at-robot ?y))
  :effect (and (at-ball ?x ?y)
               (free ?z)
               (not (carry ?z ?x))))
```

# Problem file

- Basic structure:

```
(define (problem <problem-name>)
  (:domain <domain-name>)
  <PDDL code for objects>
  <PDDL code for the initial state>
  <PDDL code for the goal description>
)
```

- **<problem-name>** is a string that identifies the planning problem
- **<domain-name>** must match the domain name in the corresponding domain file

# Objects

- **Rooms:** boyroom, girlroom
- **Balls:** blueball, greenball, pinkball, yellowball
- **TARS' arms:** left, right

## PDDL without typing

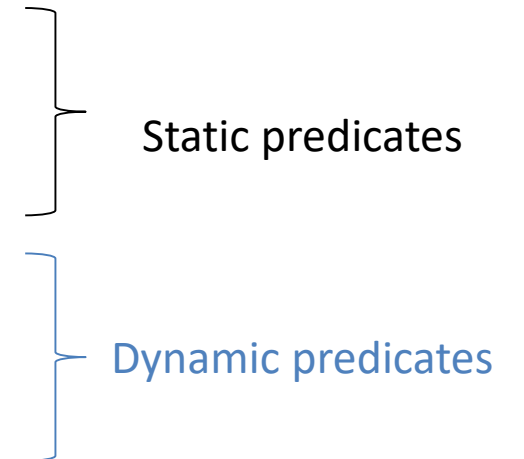
```
(:objects boyroom girlroom  
  blueball greenball  
  pinkball yellowball  
  left right)
```

## PDDL with typing

```
(:objects boyroom girlroom - room  
  blueball greenball  
  pinkball yellowball - ball  
  left right - arm)
```

# Initial state

ROOM(boyroom) **and** ROOM(girlroom) **are true.**  
BALL(blueball), ..., BALL(yellowball) **are true.**  
ARM(left), ARM(right),  
free(left) **and** free(right) **are true.**  
at-robot(boyroom), at-ball(blueball, boyroom), ...,  
at-ball(yellowball, boyroom) **are true.**



# Initial state in PDDL

## Without typing

```
(:init
  (ROOM boyroom)
  (ROOM girlroom)
  (BALL blueball)
  (BALL greenball)
  (BALL pinkball)
  (BALL yellowball)
  (ARM left)
  (ARM right)
  (free left)
  (free right)
  (at-robot boyroom)
  (at-ball blueball boyroom)
  (at-ball greenroom boyroom)
  (at-ball pinkball boyroom)
  (at-ball yellowball boyroom))
```

## With typing

```
(:init
  (free left)
  (free right)
  (at-robot boyroom)
  (at-ball blueball boyroom)
  (at-ball greenroom boyroom)
  (at-ball pinkball boyroom)
  (at-ball yellowball boyroom))
```

# Goal description

`at-ball(blueball, girlroom), ...,`  
`at-ball(yellowball, girlroom)` **must be true.**  
**Everything else is not important.**

## In PDDL

```
(:goal (and (at-ball blueball girlroom)
             (at-ball greenball girlroom)
             (at-ball pinkball girlroom)
             (at-ball yellowball girlroom))
```

# Notes (I)

- In actions, we specify what changes, and if something is not mentioned, it stays the same
- Variables in action effects must also be in action preconditions
- No explicit time terms in actions, instead time is explicitly represented
  - Preconditions always refer to time  $t$
  - Effects always refer to time  $t+1$

# Notes (2)

- The initial state is a conjunction of predicates
- Predicates not explicitly mentioned in the state are false
- The goal description is a conjunction of positive and negative literals
- In the goal description, both ground predicates and those containing variables are treated as existentially quantified



# Notes (3)

- Mostly, in your projects, you can avoid using the second level of PDDL → Add abstract meaning in the preprocessing
  - Example:
    - Precondition:  $\text{Temperature} > 30\text{ }^{\circ}\text{C}$  → Temperature is high

# Preconditions can do more

- Preconditions can be **universally quantified**

```
(forall (?x1 - type1 ... ?xn - typen)  
      <condition>)
```

- Preconditions support **arithmetic operations** (see Fox and Long, 2003)

# Effects can do more

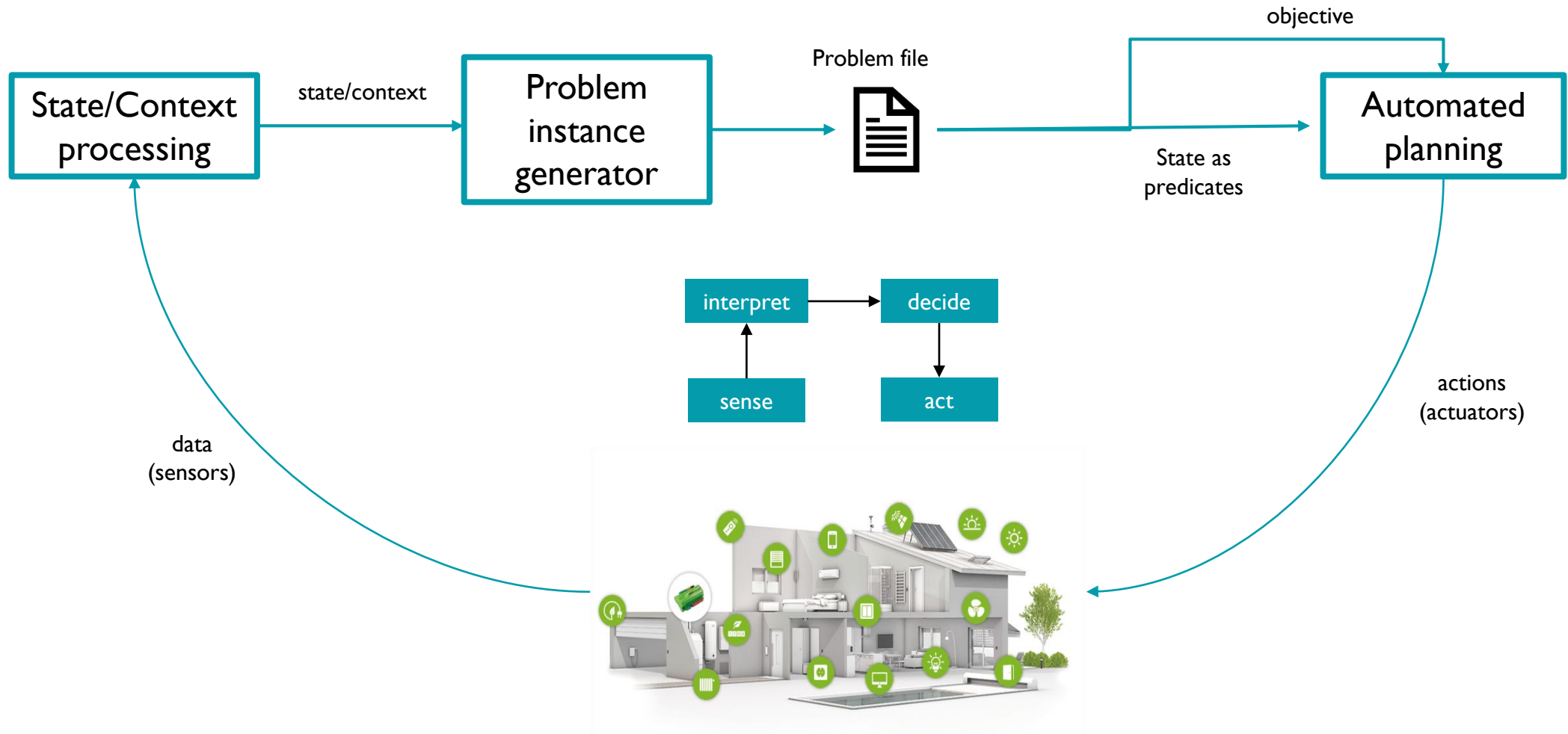
- Effects can be **universally quantified**

```
(forall (?x1 ... ?xn)  
  <effect>)
```

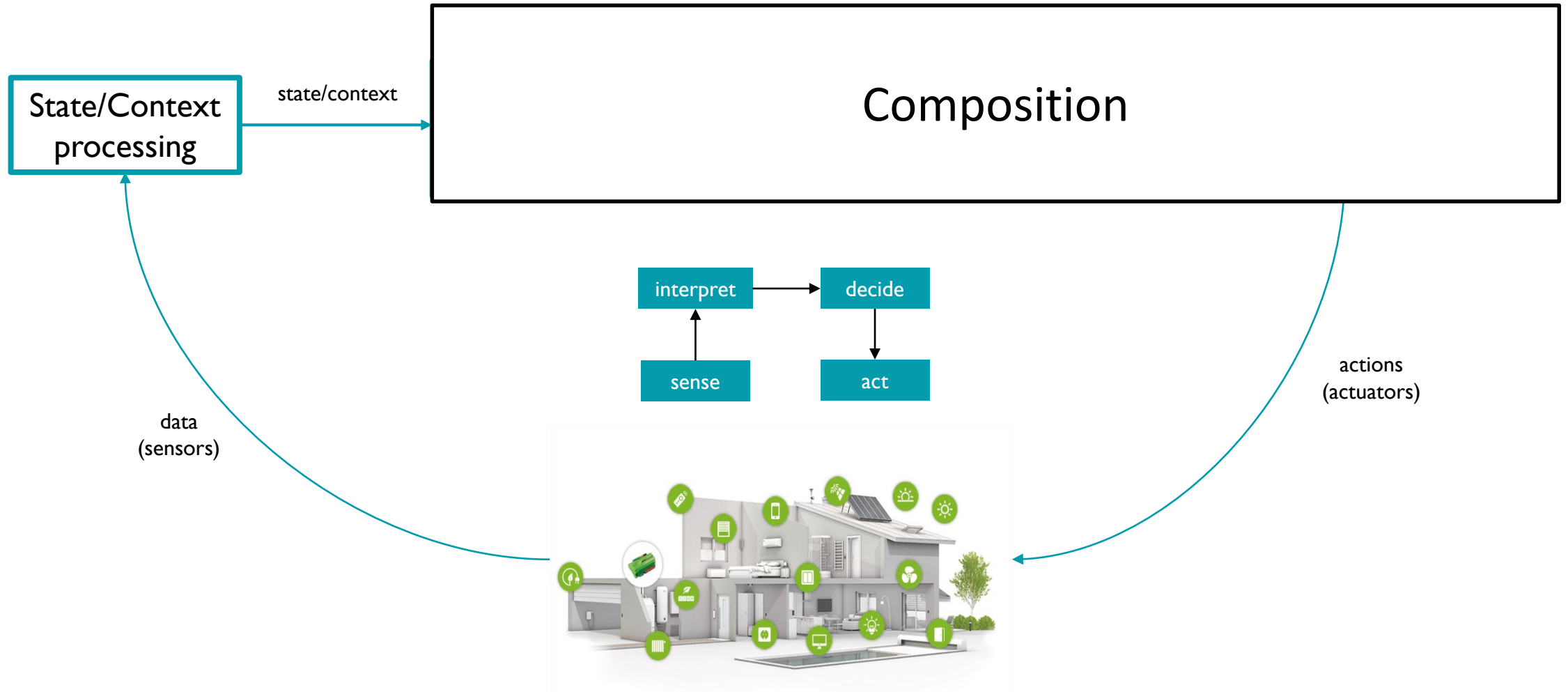
- Effects can be **conditional**

```
(when <condition>  
  <effect>)
```

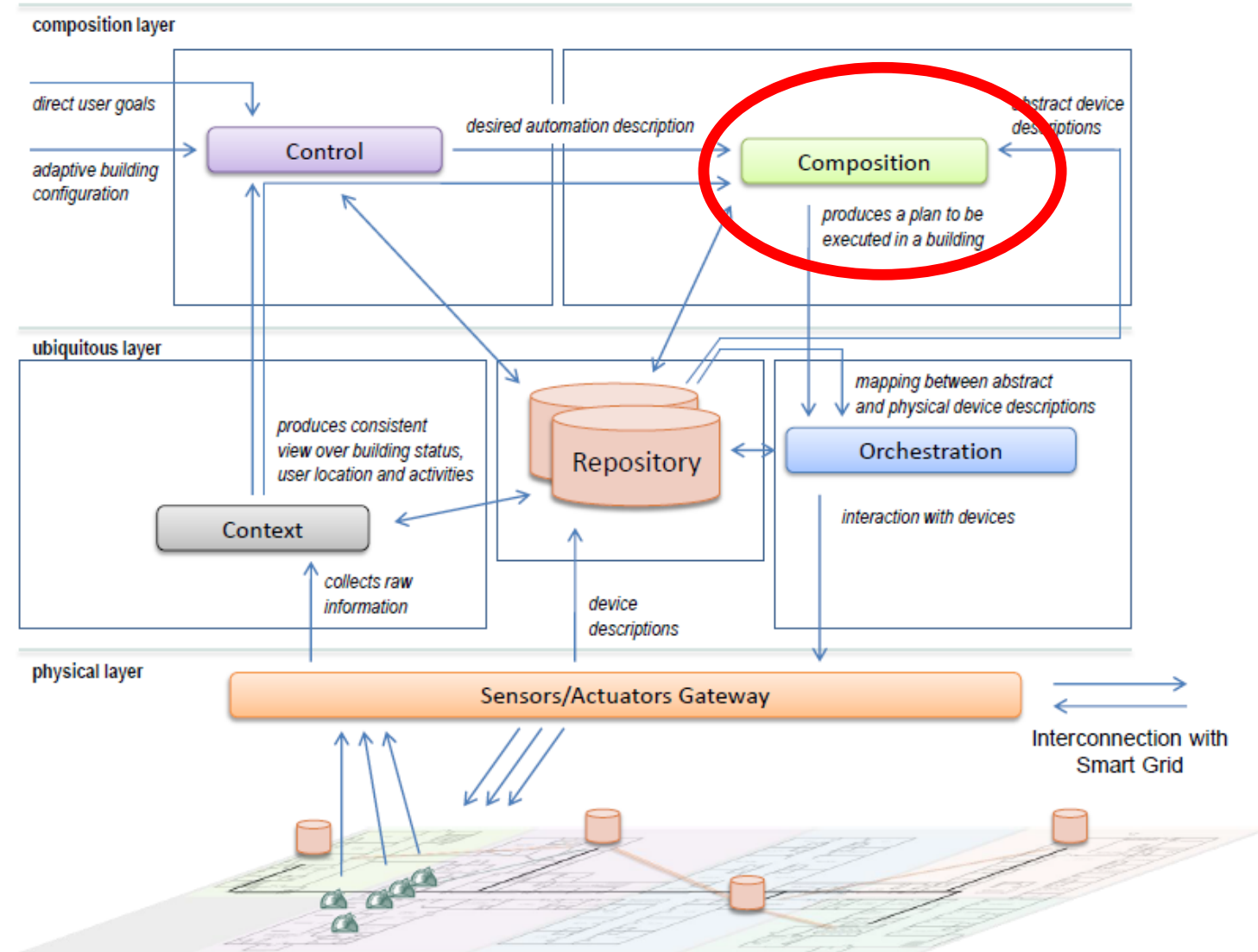
# How to put the pieces together?



# How to put the pieces together?



# How to put the pieces together?



# Exercise

- Write the domain file in PDDL
  - The “container-truck-car” domain has a set of cities and some cities are adjacent. Additionally, some cities have shortcuts that connect them. We have containers that are located in different cities and can be moved from a city to another by two means of transportation, cars and trucks. A car can only move between adjacent cities, whereas a truck can move from one city to another in case there is a shortcut connecting these two cities. Containers can be loaded/unloaded to/from the cars and trucks.

# Exercise

- Write the problem file in PDDL
  - We have 5 cities, such that the following pairs of cities are adjacent: (city1,city3), (city3,city4), (city4,city2), and (city4,city5). city1 and city4 have a shortcut that connects them. Similarly, city5 and city2 have a shortcut between them. We have two cars, a truck and two containers. Initially, the cars, the truck and the containers are in city1. The goal is to move the first container to city4 and the second container to city2. Additionally, in the final state, we don't want the two trucks to be in the same city.



# References

- Aiello, M. and Georgievski, I. Artificial Intelligence Planning in Planning for Pervasive Systems, Lecture Notes, 2019.
- Fox, M. and Long, D. (2003) PDDL2.1 :An Extension to pddl for Expressing Temporal Planning Domains, Journal of Artificial Intelligence Research 20 (2003), pp. 61-124.
- Helmert, M. An Introduction to PDDL. Online tutorial.
- Hoffmann, J. PDDL – How to Explain Your Problems to a Computer, Automatic Planning Course Slides, Saarland University, 2018.

